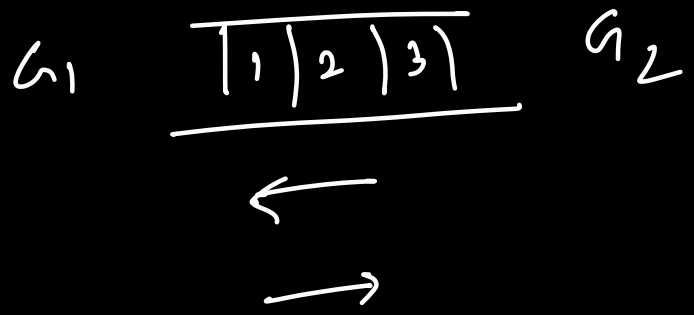
Question: Ping Pong

1. Create a function `ping`:
   - Accepts a **send-only channel**
   - Sends a given message into the channel
2. Create a function `pong`:
   - Accepts:
     - One **receive-only channel**
     - One **send-only channel**
   - Receives a message from the first channel
   - Sends the same message to the second channel
3. In `main`:
   - Create two buffered channels:
     - `pings`
     - `pongs`
   - Call `ping` to send a message
   - Call `pong` to transfer the message
   - Print the final output from `pongs`

n = 3



B	Un
→ M ✓	M X
→ Non-B	Blocki

Wait Groups

Go Concurrency

main() {

→

→ go sum()

}

1. schedule ?

2. Execution ?

Wait group

- Pain Point or Why do we need Wait Group
- What is Wait group?
- Important Functions
  - *wg.Add()*
  - *wg.Done()*
  - *wg.Wait()*
- When to use?

Wait group: Remember following points

- Why?
  - *Ensures main goroutine **waits for multiple goroutines** to finish*
  - *Prevents:*
    - Premature program exit
    - Incomplete concurrent execution
- What is Wait group?
  - *A synchronization primitive from **sync package***
  - *Maintains a **counter of active goroutines***
  - *Main goroutine blocks until counter becomes 0*
- When to use?
  - *Parallel API calls*
  - *Batch processing*
  - *Worker pools (basic coordination)*

Syntax

```go
var wg sync.WaitGroup
```

Exercise

1. Print 1 to 5 using goroutine
2. Launch 3 goroutines that print "Task N done". main() waits for all.
3. Launch 5 concurrent workers using WaitGroup.Go and ensure the main function waits until all workers complete execution.
4. 3 workers process 10 jobs. Each worker prints "Worker X processing job Y".

Worker

Exercise:

Launch 5 concurrent workers using WaitGroup.Go and ensure workers complete execution.

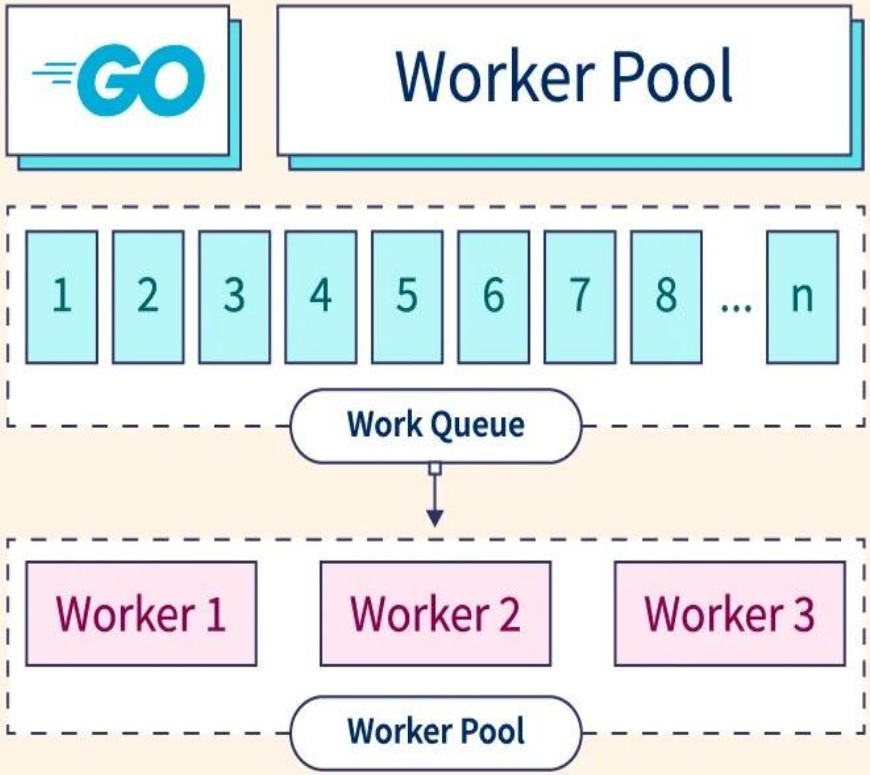
```go
wg.Go(func() {
    worker(i)
})
```

Waitgroup.Go()

- Why **waitgroup.Go**?
  - *safe wrapper over **Add + go + Done***
- It is a built-in convenience method that combines:
  - *wg.Add(1)*
  - *starting a goroutine*
  - *defer wg.Done()*

Worker Pool

- Producer
- Task Queue
- Worker



Wait Group

Note: Only for Understanding

- Fan-out
- Fan-out and Fan-in